

4-2장

2101080 정진용

1.

- MPEG-4

MPEG-4는 ISO(국제표준화기구)와 MPEG(Motion Picture Experts Group)에서 제정한 멀티미디어 압축 표준으로, 영상과 음성을 함께 효율적으로 압축할 수 있는 기술입니다. 높은 압축률과 좋은 화질을 제공하며, 인터넷 스트리밍, 모바일 기기, 디지털 방송 등 다양한 분야에서 널리 사용됩니다.

- DivX

DivX는 MPEG-4 기반으로 개발된 상용 비디오 코덱으로, 고화질의 영상을 상대적으로 작은 용량으로 저장할 수 있도록 설계되었습니다. 초창기에는 인터넷을 통한 영화 파일 공유 등에 많이 사용되었으며, DVD 및 블루레이 기기에도 지원되는 등 범용적으로 활용되었습니다.

- Xvid

Xvid는 DivX의 상용 라이선스 모델에 대항해 만들어진 오픈소스 MPEG-4 코덱입니다. 무료로 사용할 수 있고 다양한 운영체제에서 호환되기 때문에, 고화질 영상 압축이 필요하지만 상용 코덱을 피하고 싶은 사용자들에게 적합한 대안으로 평가받습니다.

- H.264 (AVC)

H.264는 고효율 비디오 압축 기술로, MPEG-4의 하위 표준 중 하나이며 현재 가장 널리 사용되는 코덱입니다. 동일 화질에서 MPEG-2보다 2배 이상의 압축률을 제공하며, 유튜브, 넷플릭스, 디지털 방송, CCTV 등 다양한 매체에서 표준으로 활용됩니다.

- H.265 (HEVC)

H.265는 H.264의 후속 규격으로, 고화질 영상을 더욱 작은 용량으로 저장할 수 있도록 개발된 고효율 비디오 코덱입니다. 4K, 8K와 같은 초고해상도 영상에 최적화되어 있으며, 최대 50%까지 데이터 용량을 줄일 수 있어 UHD 방송, 고화질 스트리밍 서비스에서 각광받고 있습니다.

2.

(ANSWER)

코드:

```
// **
// 제 목: ● 코드4-3의 동영상(stopwatch.avi)을 R,G,B값 모두 100만큼 증가시킨 후 원본과 결과영상을 모두 출력하라.
// 제 목: ● 원본 영상과 처리 후 영상을 분리하여 저장할 것
// 제 목: ● 무한루프로 작성하고 q or Q를 누르면 종료하도록 할 것
// 날 짜 : 2025년 4월 08일
// 작성자 : 2101080 정진용
// **
#include <opencv2/opencv.hpp> // OpenCV의 핵심 기능을 포함하는 헤더 파일
#include <iostream> // 표준 입출력 사용을 위한 헤더 파일
using namespace cv; // OpenCV 네임스페이스 사용
using namespace std; // 표준 C++ 네임스페이스 사용

int main(void)
{
    VideoCapture cap("stopwatch.avi"); // "stopwatch.avi"라는 비디오 파일을 열어서 cap 객체에 연결
    if (!cap.isOpened()) { // cap이 정상적으로 비디오 파일을 열었는지 확인
        cerr << "Video open failed!" << endl; // 비디오 파일 열기에 실패했을 경우 에러 메시지 출력
        return -1; // 프로그램 비정상 종료 (-1 반환)
    }

    int w = cvRound(cap.get(CAP_PROP_FRAME_WIDTH)); // 비디오의 프레임 가로 너비를 가져와 반올림 후 정수형으로 저장
    int h = cvRound(cap.get(CAP_PROP_FRAME_HEIGHT)); // 비디오의 프레임 세로 높이를 가져와 반올림 후 정수형으로 저장
    double fps = cap.get(CAP_PROP_FPS); // 초당 프레임 수(Frame Per Second)를 가져와 저장
    int delay = cvRound(1000 / fps); // 한 프레임당 지연 시간(ms)을 계산 (1000ms / fps)
    int fourcc = VideoWriter::fourcc('D', 'I', 'V', 'X'); // 'DIVX' 코덱을 fourcc 코드로 변환 (영상 저장 시 압축 방식 지정)

    VideoWriter OriginalVideo("2번 영상처리 전.avi", fourcc, fps, Size(w, h)); // 처리 전(original) 영상을 저장할 비디오 파일 생성, 코덱 및 프레임 설정 포함
    VideoWriter EditedVideo("2번 영상처리 후.avi", fourcc, fps, Size(w, h)); // 처리 후(brightened) 영상을 저장할 비디오 파일 생성, 같은 설정 사용

    Mat frame, bright; // 원본 영상 프레임(frame)과 밝기 조절된 영상(bright)을 저장할 Mat 객체 선언

    while (true) {
        cap >> frame; // cap 객체에서 한 프레임을 읽어와 frame에 저장
        if (frame.empty()) { // 프레임이 비어있으면 (즉, 영상 끝이면)
            cerr << "frame error!" << endl; // 에러 메시지 출력 후
            break; // 반복문 종료
        }

        bright = frame + Scalar(100, 100, 100); // 각 픽셀의 BGR 값을 100씩 증가시켜 밝기 조절

        OriginalVideo << frame; // 원본 영상을 파일에 저장
        EditedVideo << bright; // 밝기 조절된 영상을 파일에 저장

        imshow("frame", frame); // 원본 프레임을 윈도우에 출력
        imshow("bright", bright); // 밝기 조절된 프레임을 윈도우에 출력

        int a = waitKey(delay); // delay(ms)만큼 기다리며 키보드 입력 감지
        if (a == 'q' || a == 'Q') break; // 'q' 또는 'Q' 키를 누르면 반복 종료
    }

    return 0; // 프로그램 정상 종료
}
```

콘솔: 동영상 파일로 제출하겠습니다.

3.

(ANSWER)

코드:

```
// *****
// 제 목:   ● 코드4-3의 동영상(stopwatch.avi)을 아래 그림처럼 라인을 그린 후 원본과 결과영상을 모두 출력하라.
// 제 목:   ● 출력영상은 동영상파일로(output.avi) 저장하라->동영상을 재생하여 결과 확인할 것
// 제 목:   ● 원본 영상과 처리 후 영상을 분리하여 저장할 것
// 제 목:   ● 무한루프로 작성하고 q or Q를 누르면 종료하도록 할 것
// 날 자 : 2025년 4월 08일
// 작성자 : 2101080 정진용
// *****
#include <opencv2/opencv.hpp> // OpenCV의 핵심 헤더 파일 포함
#include <iostream>           // 표준 입출력 스트림 사용을 위한 헤더
using namespace cv;          // OpenCV 네임스페이스 사용
using namespace std;         // C++ 표준 네임스페이스 사용

int main(void)
{
    VideoCapture cap("stopwatch.avi"); // "stopwatch.avi" 비디오 파일을 열고 VideoCapture 객체 cap에 연결
    if (!cap.isOpened()) {             // cap이 정상적으로 비디오 파일을 열었는지 확인
        cerr << "Video open failed!" << endl; // 비디오 파일 열기에 실패했을 경우 오류 메시지 출력
        return -1;                       // 프로그램 비정상 종료 (에러 코드 -1 반환)
    }

    int bgr[] = { 0, 0, 255 };          // 빨간색(BGR) 색상 값 (Blue=0, Green=0, Red=255)

    int w = cvRound(cap.get(CAP_PROP_FRAME_WIDTH)); // 비디오 프레임의 가로 크기(width)를 가져와 반올림한 후 정수로 저장
    int h = cvRound(cap.get(CAP_PROP_FRAME_HEIGHT)); // 비디오 프레임의 세로 크기(height)를 가져와 반올림한 후 정수로 저장
    double fps = cap.get(CAP_PROP_FPS); // 초당 프레임 수(Frame Per Second)를 가져와 저장
    int delay = cvRound(1000 / fps); // 프레임 간 딜레이(ms) 계산 (1초 = 1000ms)
    int fourcc = VideoWriter::fourcc('D', 'I', 'V', 'X'); // 압축 코덱 설정 ('DIVX'를 FOURCC 코드로 변환)

    VideoWriter OriginalVideo("3번 영상처리 전.avi", fourcc, fps, Size(w, h)); // 원본 영상을 저장할 VideoWriter 객체 생성 ("put.avi")
    VideoWriter EditedVideo("3번 영상처리 후.avi", fourcc, fps, Size(w, h)); // 편집된 영상을 저장할 VideoWriter 객체 생성 ("output.avi")

    Mat frame, line; // frame: 원본 프레임 저장, line: 선이 추가된 편집 영상 저장

    while (true) {
        cap >> frame; // 비디오에서 한 프레임을 읽어 frame에 저장
        if (frame.empty()) { // 프레임이 비어 있다면 (더 이상 읽을 프레임이 없다면)
            cerr << "frame error!" << endl; // 에러 메시지 출력
            break; // 루프 종료
        }

        frame.copyTo(line); // 원본 frame을 line에 복사 (편집 전 원본 보존)

        // 세로 빨간 선 그리기 (영상의 중앙 세로줄에)
        for (int i = 0; i < line.rows; i++) { // 행 방향 반복
            for (int j = 0; j < 3; j++) // B, G, R 채널 각각에 대해
                line.at<Vec3b>(i, line.cols / 2)[j] = bgr[j]; // 영상 중앙 세로라인에 빨간색 픽셀 설정
        }

        // 가로 빨간 선 그리기 (영상의 중앙 가로줄에)
        for (int i = 0; i < line.cols; i++) { // 열 방향 반복
            for (int j = 0; j < 3; j++) // B, G, R 채널 각각에 대해
                line.at<Vec3b>(line.rows / 2, i)[j] = bgr[j]; // 영상 중앙 가로라인에 빨간색 픽셀 설정
        }

        OriginalVideo << frame; // 원본 프레임을 "put.avi"에 저장
        EditedVideo << line; // 십자선이 추가된 프레임을 "output.avi"에 저장

        imshow("frame", frame); // 원본 영상 출력 (창 이름: "frame")
        imshow("line", line); // 편집 영상 출력 (창 이름: "line")

        int a = waitKey(delay); // delay 시간(ms)만큼 대기하며 키보드 입력 받기
        if (a == 'q' || a == 'Q') // 'q' 또는 'Q' 키를 누르면
            break; // 루프 종료
    }

    return 0; // 프로그램 정상 종료
}
```

콘솔: 동영상 파일로 제출하겠습니다.

4. (ANSWER)

코드:

```
// *****
// 제목: ● 카메라로 촬영한 영상을 동영상 파일로 저장하는 프로그램을 작성하라.
// 제목: ● Q 또는 q를 누르면 프로그램을 종료하라.
// 제목: ● 프로젝트 폴더에 생성된 파일을 재생하여 결과를 확인할 것.
// 날 자 : 2025년 4월 08일
// 작성자 : 2101080 정진용
// *****
#include <opencv2/opencv.hpp> // OpenCV의 핵심 기능들을 포함하는 헤더 파일
#include <iostream> // 표준 입출력 관련 헤더 파일
using namespace cv; // OpenCV의 네임스페이스 사용
using namespace std; // C++ 표준 네임스페이스 사용

int main(void)
{
    VideoCapture cap("myvideo.mp4"); // "myvideo.mp4" 비디오 파일을 열고 cap 객체에 연결
    if (!cap.isOpened()) { // cap이 비디오 파일을 정상적으로 열었는지 확인
        cerr << "Video open failed!" << endl; // 열기에 실패했을 경우 에러 메시지 출력
        return -1; // 프로그램 비정상 종료 (-1 반환)
    }

    int w = cvRound(cap.get(CAP_PROP_FRAME_WIDTH)); // 비디오 프레임의 가로 너비를 반올림하여 정수형으로 저장
    int h = cvRound(cap.get(CAP_PROP_FRAME_HEIGHT)); // 비디오 프레임의 세로 높이를 반올림하여 정수형으로 저장
    double fps = cap.get(CAP_PROP_FPS); // 비디오의 초당 프레임 수 (Frame Per Second) 가져오기
    int delay = cvRound(1000 / fps); // 한 프레임 당 재생 지연 시간 계산 (1초 = 1000ms)
    int fourcc = VideoWriter::fourcc('D', 'I', 'V', 'X'); // 'DIVX' 코덱을 FOURCC 포맷으로 변환

    VideoWriter OriginalVideo("4번 저장한 파일.avi", fourcc, fps, Size(w, h));
    // 처리된 비디오를 저장할 VideoWriter 객체 생성 (파일명, 코덱, 프레임속도, 프레임크기 설정)

    Mat frame; // 한 프레임을 저장할 Mat 객체 선언

    while (true) {
        cap >> frame; // cap에서 다음 프레임을 읽어와 frame에 저장
        if (frame.empty()) { // 프레임이 비어 있다면 (파일의 끝에 도달했을 경우 등)
            cerr << "frame error!" << endl; // 에러 메시지 출력
            break; // 반복문 종료
        }

        OriginalVideo << frame; // 현재 프레임을 비디오 파일로 저장
        imshow("frame", frame); // 현재 프레임을 화면에 출력 (창 이름: "frame")

        int a = waitKey(delay); // 지정된 delay(ms)만큼 대기하며 키 입력 대기
        if (a == 'q' || a == 'Q') break; // 'q' 또는 'Q'를 누르면 반복문 종료
    }

    return 0; // 프로그램 정상 종료
}
```

콘솔: 동영상 파일로 제출하겠습니다.

5. (ANSWER)

코드:

```
//5번
// *****
// 제 목 : ● 카메라로 촬영한 영상을 사용자가 원하는 시간 동안만 동영상으로 저장하는 프로그램을 작성하라.
// 제 목 : ● 원하는 시각에서 s키를 누르면 동영상 파일로 저장을 시작하고 e키를 누르면 저장을 종료하고 프로그램을 종료하는 코드를 작성하라.
// 제 목 : ● 프로젝트 폴더에 생성된 파일을 재생하여 결과를 확인할 것.
// 날 짜 : 2025년 4월 08일
// 작성자 : 2101080 정진용
// *****
#include <opencv2/opencv.hpp> // OpenCV 라이브러리의 핵심 기능들을 포함하는 헤더
#include <iostream> // C++ 입출력 스트림을 위한 헤더
using namespace cv; // OpenCV의 네임스페이스 사용
using namespace std; // 표준 네임스페이스 사용

int main(void)
{
    VideoCapture cap("myvideo.mp4"); // "myvideo.mp4" 비디오 파일을 열고 cap 객체에 연결
    if (!cap.isOpened()) { // cap 객체가 정상적으로 비디오를 열었는지 확인
        cerr << "Video open failed!" << endl; // 열기에 실패하면 에러 메시지 출력
        return -1; // 프로그램 비정상 종료 (-1 반환)
    }

    int a = 0; // 키보드 입력 값을 저장할 변수 초기화
    bool b = false; // 저장 시작 여부를 제어할 불리언 변수 (false: 저장 안 함, true: 저장 중)

    int w = cvRound(cap.get(CAP_PROP_FRAME_WIDTH)); // 프레임의 가로 너비를 가져와 반올림 후 정수로 저장
    int h = cvRound(cap.get(CAP_PROP_FRAME_HEIGHT)); // 프레임의 세로 높이를 가져와 반올림 후 정수로 저장
    double fps = cap.get(CAP_PROP_FPS); // 영상의 초당 프레임 수(Frame Per Second) 저장
    int delay = cvRound(1000 / fps); // 프레임 간 딜레이 시간(ms) 계산 (1초 = 1000ms)
    int fourcc = VideoWriter::fourcc('D', 'I', 'V', 'X'); // 'DIVX' 코덱을 FOURCC 포맷으로 변환

    VideoWriter OriginalVideo("5번 저장한 파일.avi", fourcc, fps, Size(w, h));
    // 결과를 저장할 VideoWriter 객체 생성 ("output.avi" 파일명, 코덱, fps, 프레임 크기 설정)

    Mat frame; // 비디오 프레임을 저장할 Mat 객체 선언

    while (true) {
        cap >> frame; // 비디오에서 한 프레임을 읽어 frame에 저장
        if (frame.empty()) { // 프레임이 비어 있으면 (끝에 도달했거나 오류 발생)
            cerr << "frame error!" << endl; // 에러 메시지 출력
            break; // 반복문 종료
        }

        imshow("frame", frame); // 현재 프레임을 창에 출력 (창 이름: "frame")

        if (b) OriginalVideo << frame; // 저장 플래그 b가 true인 경우에만 프레임을 파일로 저장

        a = waitKey(delay); // delay(ms)만큼 대기하며 키보드 입력 받기

        if (a == 's') b = true; // 's' 키를 누르면 저장 시작 (b를 true로 설정)
        if (a == 'e') break; // 'e' 키를 누르면 반복문 종료
    }

    return 0; // 프로그램 정상 종료
}
```

콘솔: 동영상 파일로 제출하겠습니다.